BTK AKADEMİ

3. Gün | Kurumsal senaryolar

Yapay Zekâ Çözümleri Geliştirme Kariyer Atölyesi

Kurumsal araştırma: kaynak kontrolü

1. Soru ve arama kapsamını netleştir
2. Birincil kaynakları tercih et
3. Bulgu – kaynak – tarih bağlantısını tut
4. Çelişen bilgileri ayrıca belirt
5. Yönetici özetini insan kontrolünden geçir

Kaynağı olmayan iddia karar metnine dönüşmemeli.

Doküman analizi: toplantı notundan aksiyon

- Girdi: sentetik toplantı notu
- İstenen: karar, aksiyon, sorumlu, tarih
- Eksik sorumlu/tarih için “belirsiz” yaz
- Çıktı: eylem listesi + açık sorular

Modelin görevi eksik bilgiyi tamamlamak değil, görünür kılmaktır.

Raporlama ve veri analizi için prompt

- “Aşağıdaki özet tabloda haftalık eğilimleri bul.”
- “Farkları yüzdeyle ve baz değerle göster.”
- “Sayılarda tutarsızlık varsa işaretle.”
- “Eksik veri için kesin sonuç çıkarma.”

Sayısal sonuç, mümkünse kod ve veriyle yeniden doğrulanmalıdır.

Tekrarlayan iş: e-postayı yapılandırılmış kayda dönüştür

1

Müşteri e-postası

2

Talep sınıfı ve gerekli alanları ayıkla

3

Eksik bilgi varsa taslak yanıt hazırla

4

İnsanın onayladığı kaydı sisteme aktar

Karar destek ve otomatik icrayı ayrı tasarla.

BTK AKADEMİ

3. Gün | Uygulama

Yapay Zekâ Çözümleri Geliştirme Kariyer Atölyesi

Uygulama 1: güvenli destek sınıflandırması

UYGULAMA ADIMLARI

1. Kategori sözlüğünü ve çıktı şemasını yaz
2. Üç farklı sentetik destek kaydını dene
3. Prompt injection içeren kayıt ekle
4. Sonuçları doğruluk ve güvenlik yönünden karşılaştır

BEKLENEN ÇIKTI

Üç kayıt için şema uyumu ve güvenlik kontrol listesi; hatalı alanlar işaretli.

Girdi değişince yapı ve güvenlik davranışı korunmalı.

Uygulama 2: kurumsal prompt şablonu

UYGULAMA ADIMLARI

1. Rapor / toplantı özeti / destek talebinden birini seç
2. Rol, görev, bağlam, kısıt ve çıktı biçimini yaz
3. Birinci çıktıdaki eksikleri kaydet
4. İki revizyonla sonuçları karşılaştır

BEKLENEN ÇIKTI

Yeniden kullanılabilir prompt şablonu, iki sürüm farkı ve örnek çıktı.

Prompt kütüphanesi somut test örnekleriyle birlikte saklanır.

3. günün kazanımları ve 4. güne hazırlık

- Biçim ve iş kuralları doğrulandı mı?
- Prompt injection güven sınırı anlaşıldı mı?
- Kurum senaryosu şablona dönüştü mü?
- Sonraki gün: bunu Python ve LLM API ile çalıştıracağız.

Öğrencilerin prompt çıktısını programatik kullanıma hazırladığından emin ol.

4. günün hedefi: ihtiyaçtan çalışan koda

1

İhtiyacı tarif et

2

Kod üret, çalıştır, hata ayıkla

3

Test ve gözden geçirme ile iyileştir

4

API ve yerel model üzerinden dene

BTK AKADEMİ

3. Gün | Yazılım geliştirme

Yapay Zekâ Çözümleri Geliştirme Kariyer Atölyesi

Kod asistanları: yetenekleri ve sınırları

BTK AKADEMİ

Cursor / Copilot

Kod tamamlama, dosya bağlamı, düzenleme

Claude Code ve benzerleri

Planlama, dosya okuma, araç/komut eylemleri

Aracın önerdiği kodun sorumluluğu geliştiricide kalır.

Yazılım geliştirme ajanı ile iş ajanı farkı

BTK AKADEMİ

Geliştirici ajanı

Depodaki kodu okur, düzenler, test çalıştırır.

İş süreci ajanı

Talebi sınıflandırır, dış servis çağırır, iş akışı yürütür.

Bugün yazılım geliştirme ajanı; 7. gün iş süreci ajanı.

Önce gereksinimi yaz: destek sınıflandırıcı

- Girdi: ticket_id, açıklama, zaman, birim
- Çıktı: kategori, öncelik taslağı, eksikler
- Kural: kişisel veri yok, yetkisiz işlem yok
- Kabul ölçütü: üç örnekte şema tutarlı

İyi kod talimatı bir ürün gereksiniminden başlar.

İteratif geliştirme döngüsü

1. **İhtiyaç tanımı**
2. **Kod üretimi**
3. **Çalıştırma**
4. **Hata tespiti**
5. **Düzeltme + test**
6. **İyileştirme**

Tek seferde “kusursuz kod” beklemek yerine küçük döngüler kullan.

Python ile örnek veri modeli

```python
from dataclasses import dataclass
@dataclass
class Ticket:
    ticket_id: str
    description: str
    affected_users: int
    error_code: str | None = None
```

Veri sözleşmesi açık olursa hem kod hem prompt daha kolay test edilir.

Kod incelemesi: yalnızca çalışıyor mu?

- Fonksiyonun girdi/çıktısı açık mı?
- Hatalı veya eksik kayıtta ne oluyor?
- Secret ve kişisel veri sızıntısı var mı?
- Tekrar çalıştırılabilir ve test edilebilir mi?

Code review, okunabilirliği ve güvenli davranışı da kapsar.

Refactoring: davranışı koruyarak iyileştir

BTK AKADEMİ

Önce

Sınıflandırma, API ve format kontrolü tek fonksiyonda.

Sonra

parse_ticket(), classify(), validate_output() ayrılır.

Refactoring öncesi aynı davranışı ölçen testler olmalı.

Unit test ve sınır durumları

- Geçerli ticket sınıflandırılıyor mu?
- Boş açıklama nasıl ele alınıyor?
- Tanımsız kategori reddediliyor mu?
- LLM yanıtı bozuk JSON ise ne oluyor?

Mutlu yol kadar hatalı girdiler de test kapsamına girer.

Yapay zekâ ile hata giderme: iyi hata raporu

- Beklenen davranış nedir?
- Gerçekte ne oldu? Hata metni nedir?
- Hangi kod/sürüm/ortamda tekrarlandı?
- En küçük tekrar üretilebilir örnek nedir?

Rastgele yeniden kod üretmek yerine hatanın koşulunu belirle.